

Special class task:

1). Diff between var, let and const.

var	let	const	Features
Functional	Block	Block	Scope
Allowed	Not allowed same scope	not allowed same scope	Redeclaration
Allowed	Allowed	Not allowed	Reassignment
yes initialized to undefined	Yes, but in TDZ	Yes, but in TDZ	Hoisted

In modern JavaScript, I prefer using const by default, let when values need to change and avoid var because it can create unexpected scope-related bugs.

2). Hoisting and how it works internally.

Hoisting is the JavaScript behaviour that moving the declaration to the top of the scope before its ^{code} execution.

Internal model:

Memory creation phase: the engine creates bindings for variable/functions.

Execution phase: statements run line by line

Function declarations are fully hoisted with their definitions.

3). Temporal Dead zone (TDZ).

The temporal dead zone is the period between entering scope and the actual declaration of let or const variable. During this period accessing variable ~~reference~~ results in a ReferenceError.

4). what is an Execution context? Explain Memory creation phase and Execution phase.

An Execution context is the environment ~~that~~ in which the JavaScript code can be run.

Memory Creating phase:

- ★ creates variable bindings.
- ★ Hoists function declarations.
- ★ Allocates memory for functions.

Execution phase:

- ★ Executes statements line by line.
- ★ ~~assigns~~ assigns actual values to variables.
- ★ Invokes functions and creates new execution contexts.

Additional point: Every function call creates a new execution context with its own local memory and scope chain.

5). Scope ? Global, Function and Block scope.

scope determines where a variable is accessible

Type:

Global scope:

It can be accessible anywhere in a program

Functional scope:

It can be accessible inside that function only.

Block scope:

It can be accessible inside the nearest `{ }` block (let/const).

6). Lexical scope ? scope chain works?

Lexical scope means function's accessible variables are determined by where the function is written, not where it is called

Scope chain order

Local scope



Outer scope



Global scope

If a name is not found, JavaScript throws

ReferenceError.

7. what is closure?

A closure is a feature of javascript, and ~~the~~ a closure is created when an inner function remembers and continues to access the ~~states~~ ^{variables} from ~~the~~ ^{its} outer function, even after the outer function has finished execution.

eg:

"A closure is maintaining of bank account Balance. Balance is variable remains private inside outer function of ~~Bank~~ Account, and the inner function can access and update it even after the outer function has finished execution. This helps in data encapsulation and security.

8). Callback Function? why used in JS.

The callback Function is the function that passed as an arguments and executed later.

main uses in JS.

- for asynchronous operations
- Event handling
- customizing function behaviour
- code reusability.

10). callback injection? example.

callback injection is process of passing a callback function to customize the behaviour of a reusable function.

eg:

An order processing system processes orders in the same way, but different callbacks can send emails, SMS, notifications, or generate invoices after processing.

8). why closure preserve variable even after outer function has completed execution.

Because the inner function maintains the reference of outer function's lexical environment. As long as the inner function exists, Javascript keeps those variables in memory.

11). diff callback and closure.

callback

A function passed as an argument to another function.

Focuses on execution time.

used for asynchronous operations.

A callback can be a closure.

closure

A inner function that has remember the variable of outer function.

Focuses on variable retention.

used for data encapsulation.

but not every closure is callback.

12). diff between Rest and Spread?

Rest parameter:

- collects multiple values into an array
- used in function parameters
- Rest gather values.

spread operator:

- Expands an array/object into individual elements
- used in function calls, arrays and object
- spread expands values.

13). return and console.log()?

Answer:

return

sends value back to caller

Ends function execution

used in program logic

console.log()

Display output in console.

Does not stop execution

used for debugging.

14) • what are truthy and falsy values? List all Falsy values in JavaScript.

Falsy values are values that become false when converted to Boolean.

All falsy values are:

false

0

-0

0n

""

null

undefined

Nan



→ Everything else is Truthy.

15) • Shallow copy and Deep copy.

Shallow copy.

copies only top-level properties

shares nested object references.

Syntax:

const c = { ...obj };

Deep copy.

that copies all nested properties

creates completely independent copies

Syntax:

const copy = structuredClone(obj);

16). `diff`, `map()`, `filter()`, and `reduce`.

`map()`, `filter()`, and `reduce()` are Higher Order Array methods.

Why means.

→ Takes another function as an argument, or

→ Returns the function.

These methods take a callback function as an argument, so they are higher-order methods.

Hence

`map()` - Transforms every element

`filter()` - selects elements based on condition.

`reduce()` - produces a single accumulated value.

Map - transforms, Filter - selects, reduce - accumulates

17). `forEach` and `map()`.

`forEach`

used for side effects

Returns undefined

cannot be chained

`map`

used for transformation

Returns a new array

can be chained

Use `map()` when you need a new array and `forEach()` for actions like logging or DOM updates.

18) • diff find() and filter()?

also higher order array method were

find()

returns first matching element

stops after finding match

returns element or undefined

filter()

Returns all matching elements.

checks entire array

Returns an array.

19) • diff between for..of, for..in and forEach?

These are loop iterations where it iterates methods has differ.

for..of - that iterates over values.

for..in → that iterates over property keys (index values)

forEach() → array elements via callback.

mostly

→ for..of for arrays

→ for..in for object keys

→ forEach() for array operations.

Q0). Obj Destructuring and Enhanced object literals
why used in react.

Object Destructing that extracts the object properties
into variables.

eg:
`const user = {name: "Dim", age: 21};`
`const {name, age} = user;`

Enhanced object literals:

provide shorter syntax for creating objects

`const name = "Dim";`

`const age = 21;`

`const user = {name, age};`

why react uses it heavily means.

→ cleaner props handling

→ Easier state updates

→ Improves readability

→ supports immutable programming patterns

Hence React uses them because they make component
code, cleaner, more readable and easier to maintain